```
Main()
 Read config from DB
 Setup mutex&cond
 create Threads
 While (not shutdown)
  Handle classic backupsource
 Join threads
 Cleanup
```

```
thread_tsreceiver: (1 tråd)
 Data = Recvfrom (MC)
 AddHeaderAndPutInBuffer(firstsendbuffer, data)
   Data.Header
   Data.Header.sendtime=now()
   putInBuffer(firstsendbuffer)
```

```
thread_rudp_output:  (N trådar)
 While (not shut)
   Connect
   Signal streaming phase
```

```
thread_rudp_output_streaming_phase: (1 tråd)
  Wait for signal
  While (Connected)
    Kick on sleeping reconnect threads
    If Time to send heartbeat
      send_heartbeat()
    Wait for data | heartbeattimeout
    If data in resentbuffer
      send_data_from_buffer(resentbuffer)
      move_first_element_last(resentbuffer)
    If data in mainbuffer
      send_data_from_buffer(mainbuffer)
      move_first_element_to_other_buffer(mainbuffer,resentbuffer)
    If data in firstsendbuffer
      send_data_from_buffer(firstsendbuffer)
      move_first_element_to_other_buffer(firstsendbuffer,mainbuffer)
```

```
thread_talkback_input: (N trådar)
  Connect
  While (Connected)
    Recv_data_from_connection()
    Check for heartbeat timeout
    RUDP_PACK_SETUP:
      do handshake stuff
    RUDP_PACK_CLOSE:
      Check session ID
      do close connection
    RUDP_PACK_DATAACK:
       Check session ID
       Look for buffer in mainbuffer
          Look for buffer in resentbuffer
       calculate RTT
    RUDP_PACK_HEARTBEAT:
       Check session ID
       Record heartbeat
```

RUDP_TRANSMITTER

Availablebufferelements (1st)

Network in

thread_tsreceiver: (1 tråd)

Firstsendbuffer (1st)

Network out

thread_rudp_output_streaming_phase: (1 tråd)

Mainbuffer (1 st)

Resentbuffer (1st)

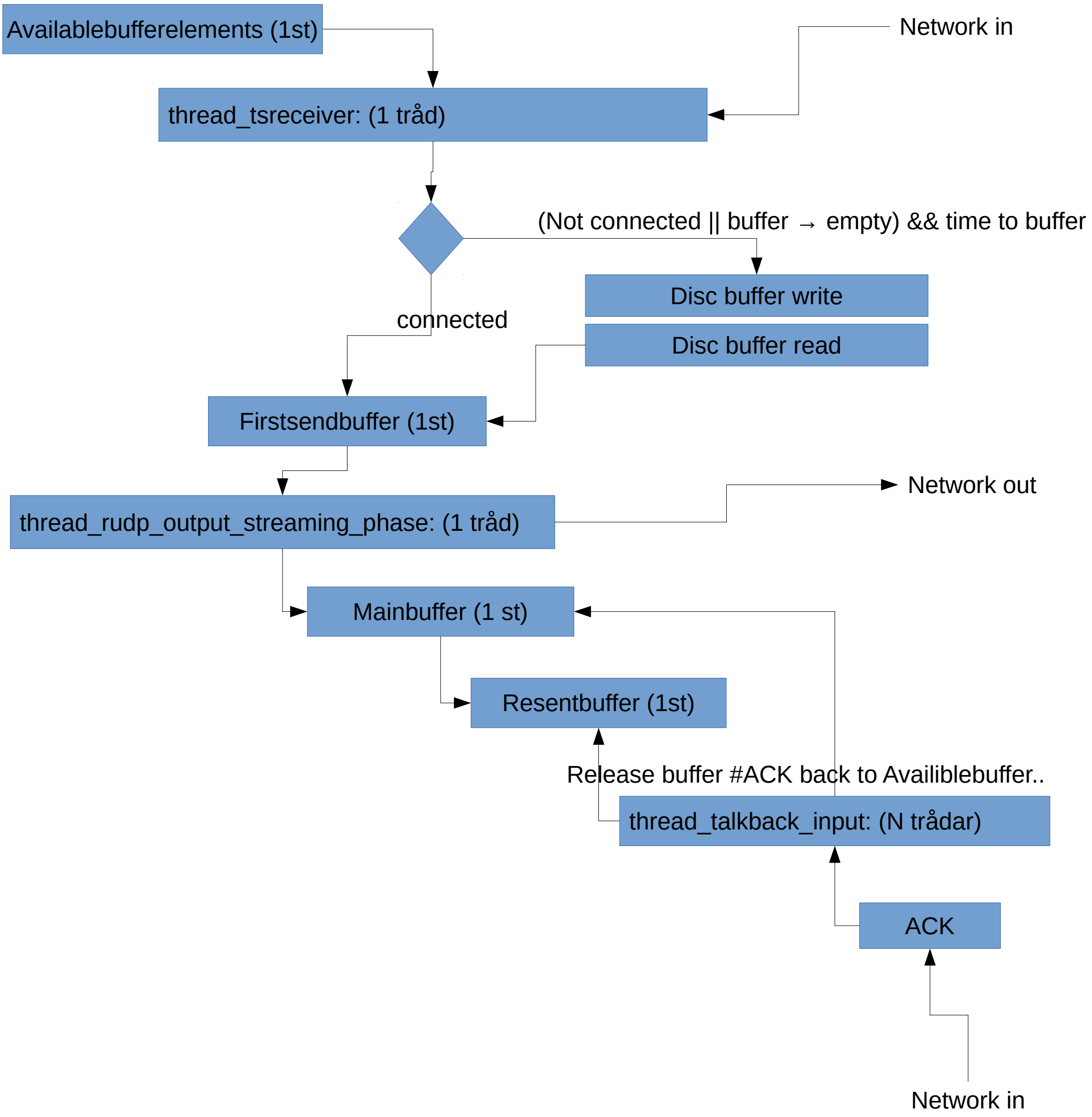
Release buffer #ACK back to Availiblebuffer..

thread_talkback_input: (N trådar)

ACK

Network in

RUDP_TRANSMITTER



RUDP_TRANSMITTER

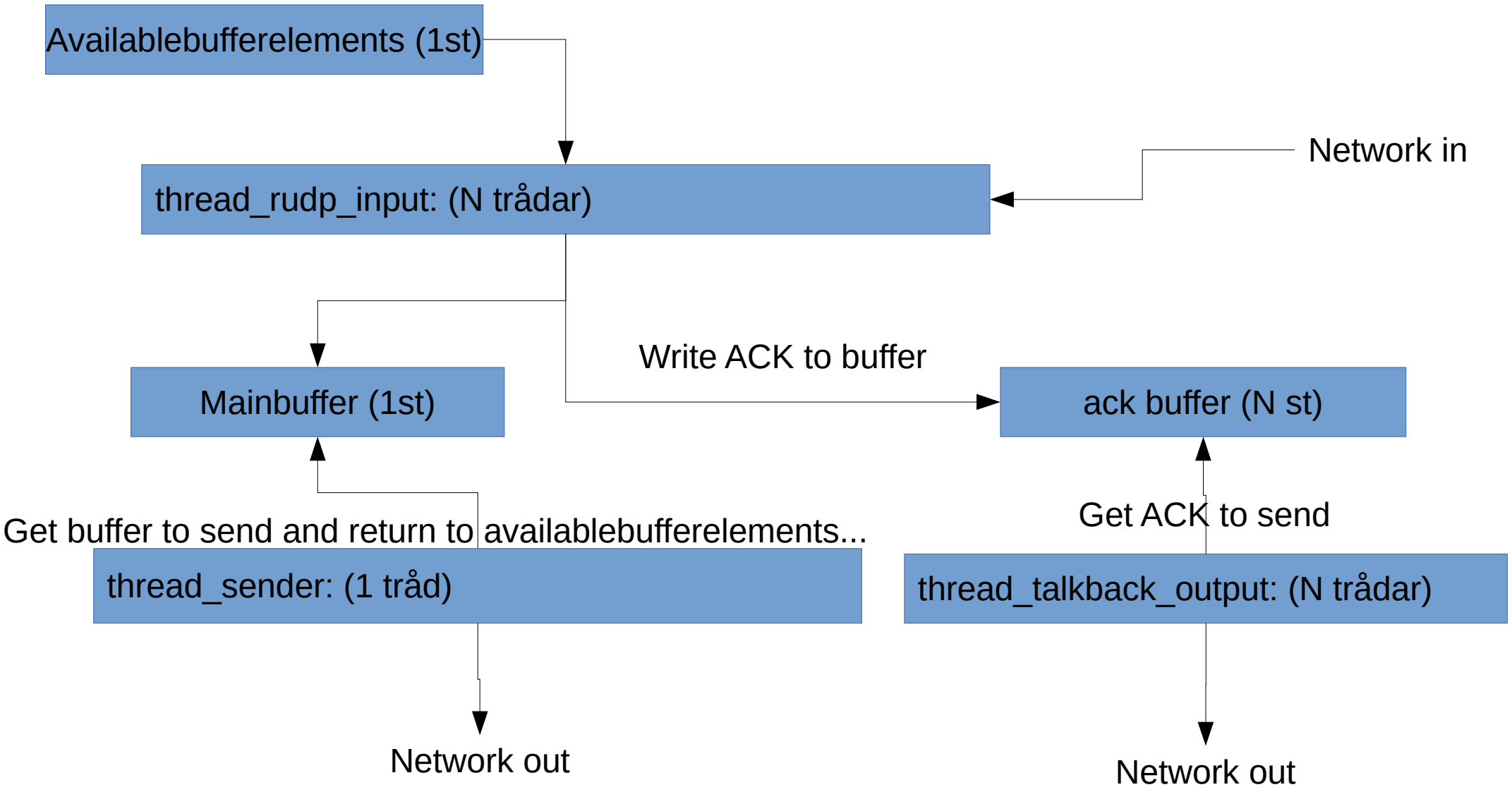
```
Main()
 Read config from DB
 Setup mutex&cond
 create Threads
 while(not shutdown)
   Print statistics
   Handle classic backup
 Join threads
 Cleanup
```

```
thread_sender: (1 tråd)
 Buffer = getTimedoutPackets(mainbuffer)
 sendto(buffer)
```

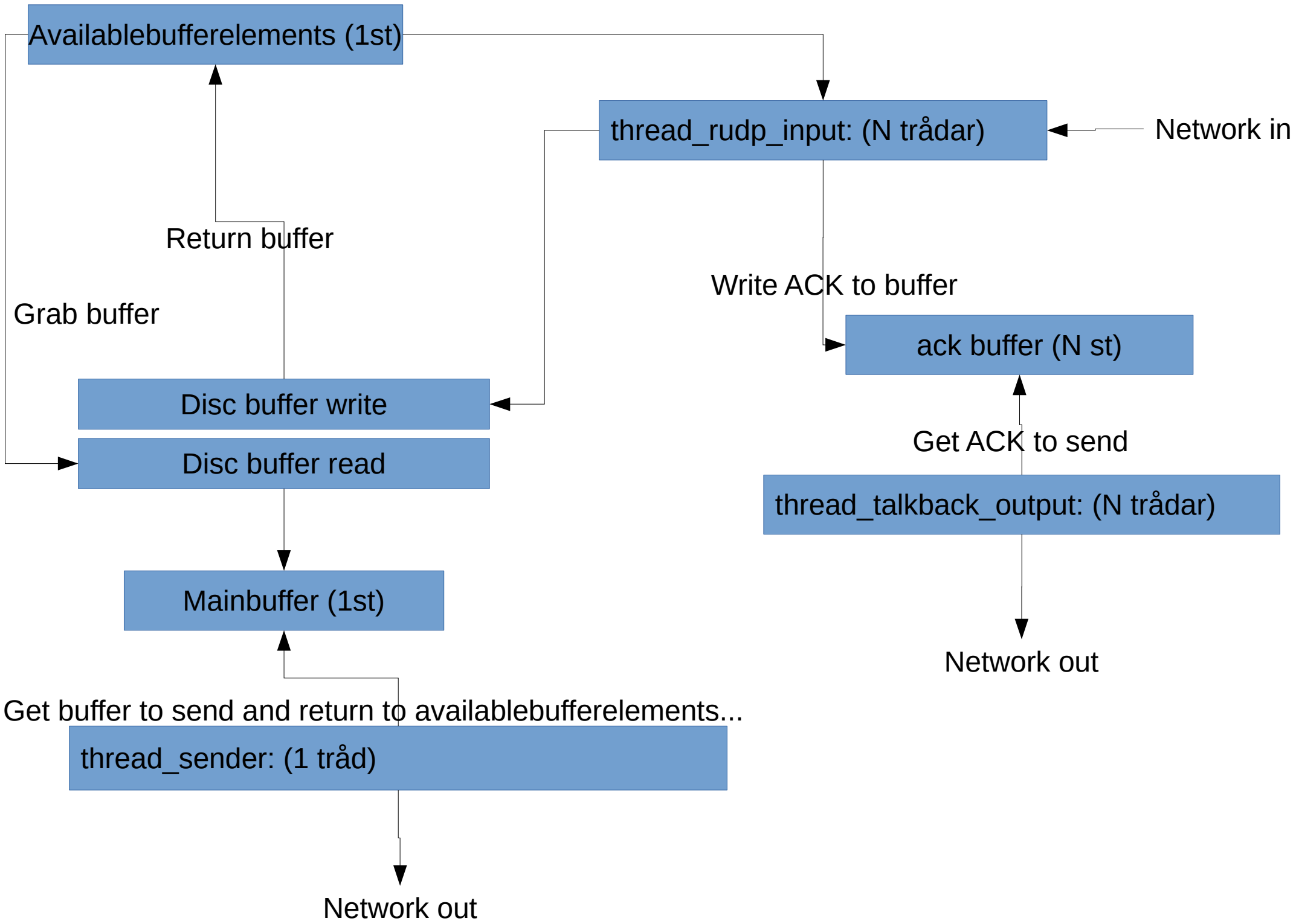
```
thread_talkback_output:  (N trådar)
 While (not shut)
   Connect
    If Time to send heartbeat
      send_heartbeat()
    If ACK to send
      send_to_connection(ACK) up to 5 ACK in a row
```

```
thread_rudp_input: (N trådar)
  Connect
  While (Connected)
    getAvailableBufferElement(availablebufferelements)
    recv_from_connection()
    Check for heartbeat timeout
    RUDP_PACK_SETUP:
       do handshake stuff
    RUDP_PACK_DATA:
       Check session id
       putInBuffer(mainbuffer)
       Do statistic stuff
    RUDP_PACK_HEARTBEAT:
        Check session ID
        Record heartbeat
    RUDP_PACK_CLOSE:
       Check session ID
       do close connection
```

RUDP_RECEIVER



RUDP_RECEIVER



Usecase:

1) Buffrar som kommer in har ”lång tid” till sändning
   => senast nedskrivna # +1 == mottagen # => skriv ner
   => Läser upp X buffrar i början och väntar på att den ska sändas
   => När skickad buffer frigörs läs en ny från disk
2) initialt finns det tid att spara på disk (gör som 1)
   => men överföringen går sönder av någon anledning
   => den börjar spela upp från disk
   => Operatör ”fixar” uppkoppling och den kommer i läget att det inte finns tid att spara på disk utan måste gå över till classic streaming läge

RUDP_RECEIVER